

Guía de Laboratorio N.º 3

Implementación de un Datamart Analítico y Creación de Cubos

1. Datos generales

Modalidad	INDIVIDUAL. Cada estudiante desarrolla, ejecuta y entrega su propio trabajo.
-----------	--

1.1. Objetivos

Al concluir este laboratorio, el estudiante será capaz de:

- Perfilar archivos de origen e identificar problemas de calidad que condicionan el diseño del datamart.
- Escribir el DDL completo de un esquema estrella, con claves subrogadas, tipos ajustados, índices y particionamiento.
- Generar por programa una dimensión tiempo con atributos de calendario del negocio.
- Cargar dimensiones aplicando el patrón de dimensión lentamente cambiante Tipo 2.
- Cargar una tabla de hechos mediante búsqueda de claves subrogadas sensible a la fecha.
- Ejecutar y documentar siete cuadros de control que demuestren la corrección de la carga.
- Resolver las seis operaciones OLAP en SQL y materializar un cubo con GROUPING SETS.
- Construir un modelo semántico sobre el datamart y validar que sus totales coinciden con el origen.

1.2. Requisitos previos

- Haber revisado el material teórico de la Semana 3, en especial los apartados de diseño lógico, diseño físico y cuadros de control.
- MySQL 8.0 o superior instalado y en ejecución, con un cliente gráfico (MySQL Workbench, DBeaver o similar).
- Conocimientos de SQL a nivel de Base de Datos II: JOIN, GROUP BY, subconsultas, funciones de agregación y DDL.
- Power BI Desktop instalado (versión gratuita) para la etapa 9. Alternativa aceptada: Tableau Public.

Si usted trabaja con otro motor

La guía está escrita para MySQL 8 porque es el motor disponible en los laboratorios de la Facultad. Puede desarrollarla en SQL Server, PostgreSQL o MariaDB: al final de cada etapa que lo requiera se indican las equivalencias. Lo que no se acepta es cambiar el diseño para evitar una dificultad — por ejemplo, eliminar el SCD Tipo 2 porque resulta trabajoso—; el diseño es el objeto de la evaluación.

1.3. Entregables

Se entrega un único archivo comprimido con el nombre Lab3_ApellidoNombre.zip, que debe contener exactamente:

N.º	Archivo	Contenido
1	dm_mercandina_ddl.sql	Creación de la base de datos, las seis dimensiones y la tabla de hechos, con tipos, claves, índices y particionamiento.
2	dm_mercandina_dim.sql	Filas especiales, generación de la dimensión tiempo y carga inicial de las dimensiones desde staging.
3	dm_mercandina_scd2.sql	Aplicación de los cambios del 1 de abril de 2026 con el patrón SCD Tipo 2.
4	dm_mercandina_carga.sql	Carga de la tabla de hechos con búsqueda de claves sensible a la fecha.
5	dm_mercandina_olap.sql	Las seis operaciones OLAP y la consulta con GROUPING SETS.
6	cuadros_control.pdf	Los siete cuadros ejecutados, con captura del resultado y una línea de interpretación por cada uno.
7	dm_mercandina.pbix	Modelo semántico con relaciones, medidas, jerarquías y la matriz de operaciones OLAP.
8	actividad_propuesta.pdf	Desarrollo de la actividad de la sección 12.

Tabla 1. Entregables del laboratorio.

Condición de aceptación

Los scripts deben ejecutarse de principio a fin, en el orden indicado, sobre una base de datos vacía y sin intervención manual. Un script que solo funciona si el docente adivina qué sentencia ejecutar primero se califica como no entregado. Verifique esto antes de comprimir: borre la base de datos, vuelva a ejecutar los cuatro scripts en orden y confirme que los cuadros siguen dando lo mismo.

2. Etapa 0: Preparación del entorno

2.1. Descarga de los archivos de origen

Descargue del Aula Virtual el archivo datos_mercandina.zip, ubicado en la carpeta de la Semana 3, y descomprímalo en una carpeta de trabajo. Se recomienda C:\lab3\datos:

Archivo	Filas	Sistema fuente	Contenido
productos.csv	386	ERP	Extracto del catálogo de productos con marca, categoría, subcategoría, precio y costo.
productos_cambios.csv	28	ERP	Reclasificación de categoría vigente desde el 01/04/2026.
tiendas.csv	34	ERP	Maestro de locales con formato, distrito, ciudad, región y área.
tiendas_cambios.csv	6	ERP	Cambio de formato Express a Market vigente desde el 01/04/2026.
clientes.csv	1 200	Fidelización	Clientes registrados con segmento, distrito y ciudad.
clientes_cambios.csv	95	Fidelización	Cambios de distrito y segmento vigentes desde el 01/04/2026.
promociones.csv	12	ERP	Campañas del semestre con tipo y periodo de vigencia.
ventas.csv	106 708	POS	Líneas de comprobante del 01/01/2026 al 30/06/2026.

Tabla 2. Archivos de origen del laboratorio.

Todos los archivos son CSV con delimitador coma, punto como separador decimal, codificación UTF-8 y una fila de encabezado. Las fechas están en formato ISO (AAAA-MM-DD).

Si no puede descargar el archivo del Aula Virtual

Los mismos archivos están publicados en el repositorio del curso. En ese caso, verifique después de la descarga que ventas.csv tenga 106 708 filas de datos más el encabezado. En Linux o macOS: `wc -l ventas.csv` debe devolver 106709. En Windows PowerShell: `(Get-Content ventas.csv).Count`. Si el número no coincide, la descarga se truncó y debe repetirla: todos los cuadros de control posteriores dependerán de esa cifra.

2.2. Verificación del motor de base de datos

Conéctese a MySQL y confirme la versión y la ruta autorizada para la carga de archivos:

```
SELECT VERSION(); -- debe ser 8.0 o superior
SHOW VARIABLES LIKE 'secure_file_priv'; -- carpeta autorizada para LOAD DATA
SHOW VARIABLES LIKE 'local_infile'; -- debe estar en ON
```

Figura 1. Verificación previa del entorno.

La variable `secure_file_priv` indica la única carpeta desde la que MySQL acepta leer archivos. Tiene tres comportamientos posibles y cada uno exige una acción distinta:

Valor devuelto	Qué significa	Qué debe hacer
Una ruta, por ejemplo C:\ProgramData\MySQL\Uploads\	Solo se puede leer desde esa carpeta.	Copie los ocho CSV a esa carpeta y use esa ruta en LOAD DATA INFILE.
Cadena vacía	No hay restricción de carpeta.	Puede usar la ruta de su carpeta de trabajo directamente.
NULL	La carga desde archivo está deshabilitada por completo.	Use LOAD DATA LOCAL INFILE con <code>local_infile</code> activado, o importe con el asistente del cliente gráfico.

Tabla 3. Interpretación de `secure_file_priv`.

Si `local_infile` aparece en OFF y necesita usar la variante LOCAL, actívela en la sesión y en el cliente:

```
-- En el servidor (requiere privilegio SUPER o SYSTEM_VARIABLES_ADMIN)
SET GLOBAL local_infile = 1;

-- En el cliente de línea de comandos, agregue el modificador:
-- mysql --local-infile=1 -u root -p

-- En MySQL Workbench: Edit > Preferences > SQL Editor >
-- marcar "Allow LOAD DATA LOCAL INFILE"
```

Figura 2. Habilitación de la carga local de archivos.

Plan alternativo si LOAD DATA sigue bloqueado

En laboratorios con permisos restringidos, use el asistente de importación del cliente: en MySQL Workbench, clic derecho sobre el esquema, Table Data Import Wizard; en DBeaver, clic derecho sobre la tabla, Import Data. Es más lento con ventas.csv (unos minutos), pero produce exactamente el mismo resultado. Deje constancia en `cuadros_control.pdf` del método que utilizó.

3. Etapa 1: Perfilado de los archivos de origen

Ningún diseño físico se decide sin haber mirado los datos. Antes de crear una sola tabla, abra los archivos y responda a las preguntas de esta etapa. El objetivo no es la exploración por sí misma, sino tomar decisiones concretas: qué tipo de dato usar, qué longitud declarar y qué problemas de calidad va a tener que absorber el proceso de carga.

3.1. Inspección inicial

Abra productos.csv, tiendas.csv y clientes.csv en un editor de texto o en una hoja de cálculo. No abra ventas.csv en Excel: con más de cien mil filas, la hoja de cálculo se vuelve lenta y, sobre todo, Excel reinterpreta los formatos de fecha y puede corromper el archivo si lo guarda. Para inspeccionar las primeras líneas use el terminal:

```
# Linux o macOS
head -5 ventas.csv
wc -l ventas.csv

# Windows PowerShell
Get-Content ventas.csv -TotalCount 5
(Get-Content ventas.csv).Count
```

Figura 3. Inspección de las primeras filas sin abrir el archivo completo.

3.2. Preguntas de perfilado

Responda por escrito en cuadros_control.pdf, sección «Perfilado». Estas respuestas justifican las decisiones de las etapas siguientes:

1. ¿Cuál es la longitud máxima del campo nombre_producto? Ese valor determina el VARCHAR que declarará, con un margen razonable.
2. ¿Qué valores distintos toma unidad_medida? ¿Y forma_pago en ventas.csv? La cardinalidad de estos campos decide si merecen dimensión propia o entran en la dimensión junk.
3. En ventas.csv, ¿qué proporción de filas tiene el campo doc_cliente vacío? ¿Qué significa esa ausencia en el negocio y a qué clave subrogada deben apuntar esas filas?
4. ¿Qué proporción de filas tiene cod_promocion vacío? ¿Es el mismo tipo de ausencia que la anterior?
5. ¿Existen filas con cantidad negativa? ¿Qué representan y qué implicación tiene para el signo de importe_netto?
6. ¿Todos los cod_producto de ventas.csv existen en productos.csv? Verifíquelo; el resultado condiciona el tipo de JOIN que escribirá en la etapa 6.

Para la última pregunta puede adelantarse y usar SQL después de cargar el área de staging, o resolverlo con una línea de terminal:

```
# Codigos de producto presentes en ventas pero ausentes del catalogo (Linux/macOS)
comm -23 \
  <(cut -d, -f4 ventas.csv | tail -n +2 | sort -u) \
  <(cut -d, -f1 productos.csv | tail -n +2 | sort -u)
```

Figura 4. Detección de códigos huérfanos antes de cargar.

Resultado esperado del perfilado

Debe encontrar siete códigos de producto que aparecen en las ventas y no existen en el catálogo del ERP, y que en conjunto afectan a algo más del 0,3 % de las líneas. Este es el caso exacto que justifica la fila «Desconocido» con clave -1: si esas líneas se descartan con un INNER JOIN, la venta total del datamart dejará de coincidir con la del sistema fuente y nadie sabrá por qué.

4. Etapa 2: Creación del esquema (dm_mercandina_ddl.sql)

Cree un archivo llamado dm_mercandina_ddl.sql y escriba en él todo el contenido de esta etapa. Ejecútelo completo al final, no sentencia por sentencia: el objetivo es que el script sea reproducible.

4.1. Base de datos y área de staging

El área de staging es una copia fiel del origen, sin transformar. Todos sus campos se declaran como texto: en esta capa no se valida ni se convierte nada, porque cualquier conversión fallida detendría la carga y perdería la trazabilidad hacia el archivo original.

```
DROP DATABASE IF EXISTS dm_mercandina;
CREATE DATABASE dm_mercandina
  DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE dm_mercandina;

-- ===== AREA DE STAGING (espejo del origen) =====
CREATE TABLE stg_producto (
  cod_producto VARCHAR(20), nombre_producto VARCHAR(200),
  marca VARCHAR(80), categoria VARCHAR(60),
  subcategoria VARCHAR(60), unidad_medida VARCHAR(15),
  es_marca_propia VARCHAR(5), precio_lista VARCHAR(20),
  costo_unitario VARCHAR(20)
) ENGINE=InnoDB;

CREATE TABLE stg_producto_cambio LIKE stg_producto;
ALTER TABLE stg_producto_cambio ADD COLUMN fecha_cambio VARCHAR(20);

CREATE TABLE stg_tienda (
  cod_tienda VARCHAR(15), nombre_tienda VARCHAR(120),
  formato VARCHAR(30), distrito VARCHAR(60),
  ciudad VARCHAR(60), region VARCHAR(60),
  area_m2 VARCHAR(20), fecha_apertura VARCHAR(20)
) ENGINE=InnoDB;

CREATE TABLE stg_tienda_cambio LIKE stg_tienda;
ALTER TABLE stg_tienda_cambio ADD COLUMN fecha_cambio VARCHAR(20);

CREATE TABLE stg_cliente (
  doc_cliente VARCHAR(20), nombre VARCHAR(120), segmento VARCHAR(30),
  distrito VARCHAR(60), ciudad VARCHAR(60), fecha_alta VARCHAR(20)
) ENGINE=InnoDB;

CREATE TABLE stg_cliente_cambio LIKE stg_cliente;
ALTER TABLE stg_cliente_cambio ADD COLUMN fecha_cambio VARCHAR(20);

CREATE TABLE stg_promocion (
  cod_promocion VARCHAR(15), nombre_promocion VARCHAR(120),
  tipo_promocion VARCHAR(40), fecha_inicio VARCHAR(20),
  fecha_fin VARCHAR(20)
) ENGINE=InnoDB;

CREATE TABLE stg_venta_pos (
  nro_ticket VARCHAR(30), fecha_emision VARCHAR(20),
  cod_tienda VARCHAR(15), cod_producto VARCHAR(20),
  doc_cliente VARCHAR(20), cod_promocion VARCHAR(15),
  tipo_comprobante VARCHAR(20), forma_pago VARCHAR(25),
  canal VARCHAR(20), flag_devolucion VARCHAR(5),
  cantidad VARCHAR(20), precio_unitario VARCHAR(20),
```

```
costo_unitario VARCHAR(20), descuento VARCHAR(20),
importe_neto VARCHAR(20),
KEY ix_stg_fecha (fecha_emision),
KEY ix_stg_prod (cod_producto)
) ENGINE=InnoDB;
```

Figura 5. Base de datos y tablas de staging.

Por qué el staging lleva índices y las tablas de origen no

Los dos índices de stg_venta_pos no están para consultar: están para que la carga de la tabla de hechos, que hará un join contra cuatro dimensiones sobre cien mil filas, no degenera en un recorrido completo por cada fila. Es una decisión de diseño físico, y su justificación es exactamente la que se estudió en la teoría: el índice acelera la lectura a cambio de ralentizar la escritura, y aquí la lectura ocurre una vez pero contra un volumen grande.

4.2. Dimensiones

Continúe en el mismo archivo. Observe tres decisiones que debe reproducir en todas las dimensiones con historia: la clave subrogada es autoincremental, la clave natural se indexa junto al indicador de vigencia (porque la búsqueda de la etapa 6 usará ambos campos), y los importes nunca son FLOAT.

```
-- ===== DIMENSIONES =====
CREATE TABLE dim_tiempo (
    tiempo_key INT NOT NULL, -- AAAAMMDD
    fecha DATE NOT NULL,
    anio SMALLINT NOT NULL,
    trimestre TINYINT NOT NULL,
    mes TINYINT NOT NULL,
    nombre_mes VARCHAR(12) NOT NULL,
    anio_mes CHAR(7) NOT NULL,
    dia_mes TINYINT NOT NULL,
    dia_semana TINYINT NOT NULL,
    nombre_dia VARCHAR(10) NOT NULL,
    es_fin_semana TINYINT(1) NOT NULL,
    es_dia_pago TINYINT(1) NOT NULL,
    PRIMARY KEY (tiempo_key),
    UNIQUE KEY ux_tiempo_fecha (fecha)
) ENGINE=InnoDB;

CREATE TABLE dim_producto (
    producto_key INT NOT NULL AUTO_INCREMENT,
    producto_id VARCHAR(20) NOT NULL, -- clave natural
    nombre_producto VARCHAR(200) NOT NULL,
    marca VARCHAR(80) NOT NULL,
    categoria VARCHAR(60) NOT NULL, -- SCD Tipo 2
    subcategoria VARCHAR(60) NOT NULL, -- SCD Tipo 2
    unidad_medida VARCHAR(15) NOT NULL,
    es_marca_propia TINYINT(1) NOT NULL DEFAULT 0,
    precio_lista DECIMAL(12,2) NULL,
    costo_estandar DECIMAL(12,2) NULL,
    fecha_inicio_vig DATE NOT NULL,
    fecha_fin_vig DATE NOT NULL DEFAULT '9999-12-31',
    es_vigente TINYINT(1) NOT NULL DEFAULT 1,
    fecha_carga DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (producto_key),
    KEY ix_prod_natural (producto_id, fecha_inicio_vig, fecha_fin_vig),
    KEY ix_prod_categoria (categoria, subcategoria)
) ENGINE=InnoDB;

CREATE TABLE dim_tienda (
    tienda_key INT NOT NULL AUTO_INCREMENT,
    tienda_id VARCHAR(15) NOT NULL,
    nombre_tienda VARCHAR(120) NOT NULL,
    formato VARCHAR(30) NOT NULL, -- SCD Tipo 2
    distrito VARCHAR(60) NOT NULL,
    ciudad VARCHAR(60) NOT NULL,
    region VARCHAR(60) NOT NULL,
    area_m2 DECIMAL(10,2) NOT NULL, -- SCD Tipo 2
    fecha_apertura DATE NULL, -- SCD Tipo 0
)
```

```
fecha_inicio_vig DATE NOT NULL,
fecha_fin_vig DATE NOT NULL DEFAULT '9999-12-31',
es_vigente TINYINT(1) NOT NULL DEFAULT 1,
PRIMARY KEY (tienda_key),
KEY ix_tienda_natural (tienda_id, fecha_inicio_vig, fecha_fin_vig)
) ENGINE=InnoDB;

CREATE TABLE dim_cliente (
  cliente_key INT NOT NULL AUTO_INCREMENT,
  cliente_id VARCHAR(20) NOT NULL,
  nombre VARCHAR(120) NOT NULL,
  segmento VARCHAR(30) NOT NULL, -- SCD Tipo 2
  distrito VARCHAR(60) NOT NULL, -- SCD Tipo 2
  ciudad VARCHAR(60) NOT NULL, -- SCD Tipo 2
  fecha_alta DATE NULL, -- SCD Tipo 0
  fecha_inicio_vig DATE NOT NULL,
  fecha_fin_vig DATE NOT NULL DEFAULT '9999-12-31',
  es_vigente TINYINT(1) NOT NULL DEFAULT 1,
  PRIMARY KEY (cliente_key),
  KEY ix_cli_natural (cliente_id, fecha_inicio_vig, fecha_fin_vig)
) ENGINE=InnoDB;

CREATE TABLE dim_promocion (
  promocion_key INT NOT NULL AUTO_INCREMENT,
  promocion_id VARCHAR(15) NOT NULL,
  nombre_promocion VARCHAR(120) NOT NULL,
  tipo_promocion VARCHAR(40) NOT NULL,
  fecha_inicio DATE NULL,
  fecha_fin DATE NULL,
  PRIMARY KEY (promocion_key),
  KEY ix_promo_natural (promocion_id)
) ENGINE=InnoDB;

-- Dimension JUNK: agrupa indicadores de baja cardinalidad
CREATE TABLE dim_transaccion (
  transaccion_key INT NOT NULL AUTO_INCREMENT,
  tipo_comprobante VARCHAR(20) NOT NULL,
  forma_pago VARCHAR(25) NOT NULL,
  canal VARCHAR(20) NOT NULL,
  es_devolucion TINYINT(1) NOT NULL,
  PRIMARY KEY (transaccion_key),
  UNIQUE KEY ux_trans (tipo_comprobante, forma_pago, canal, es_devolucion)
) ENGINE=InnoDB;
```

Figura 6. DDL de las seis dimensiones.

4.3. Tabla de hechos

Note que la clave primaria es exactamente el grano declarado: una línea de comprobante queda identificada por la fecha, el producto, la tienda y el número de ticket. Declarar así la clave primaria no es una formalidad: es el mecanismo que impedirá físicamente que una carga duplicada pase inadvertida.

```
CREATE TABLE fact_venta (
  tiempo_key INT NOT NULL,
  producto_key INT NOT NULL,
  tienda_key INT NOT NULL,
  cliente_key INT NOT NULL,
  promocion_key INT NOT NULL,
  transaccion_key INT NOT NULL,
  nro_ticket VARCHAR(30) NOT NULL, -- dimension degenerada
  cantidad DECIMAL(12,3) NOT NULL,
  importe_venta DECIMAL(18,2) NOT NULL,
  importe_costo DECIMAL(18,2) NOT NULL,
  importe_descuento DECIMAL(18,2) NOT NULL DEFAULT 0,
  PRIMARY KEY (tiempo_key, producto_key, tienda_key, nro_ticket),
  KEY ix_fact_producto (producto_key),
  KEY ix_fact_tienda (tienda_key),
  KEY ix_fact_cliente (cliente_key),
  KEY ix_fact_promo (promocion_key),
  KEY ix_fact_ticket (nro_ticket)
)
ENGINE=InnoDB
PARTITION BY RANGE (tiempo_key) (
```

```
PARTITION p2025 VALUES LESS THAN (20260101),
PARTITION p2026q1 VALUES LESS THAN (20260401),
PARTITION p2026q2 VALUES LESS THAN (20260701),
PARTITION p2026q3 VALUES LESS THAN (20261001),
PARTITION p2026q4 VALUES LESS THAN (20270101),
PARTITION pmax VALUES LESS THAN MAXVALUE
);
```

Figura 7. DDL de la tabla de hechos, particionada por trimestre.

Una restricción de MySQL que debe conocer

En MySQL, toda columna de la clave de partición debe formar parte de cada clave única de la tabla. Por eso tiempo_key encabeza la clave primaria: si intentara particionar por tiempo_key con una clave primaria que no lo incluyera, MySQL rechazaría la sentencia. Además, InnoDB con particionamiento no admite claves foráneas; por eso este script no las declara y la integridad se garantiza en la carga y se comprueba con los cuadros de la etapa 7.

En SQL Server y PostgreSQL puede mantener las claves foráneas junto al particionamiento. Si trabaja en esos motores, decláralas y menciónelo en su informe.

Ejecute ahora el script completo y verifique que las trece tablas existen:

```
SHOW TABLES; -- 6 de staging + 6 dimensiones + 1 hecho
SELECT COUNT(*) FROM information_schema.partitions
WHERE table schema = 'dm mercandina' AND table name = 'fact venta'; -- 6
```

Figura 8. Verificación del esquema creado.

5. Etapa 3: Carga del área de staging

Cree el archivo dm_mercandina_dim.sql. Comience cargando los ocho archivos en las tablas de staging. Sustituya /ruta/a/datos/ por la ruta real de su carpeta; en Windows use barras normales o barras invertidas dobles, por ejemplo C:/lab3/datos/.

```
USE dm_mercandina;

LOAD DATA LOCAL INFILE '/ruta/a/datos/productos.csv'
INTO TABLE stg_producto
FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;

LOAD DATA LOCAL INFILE '/ruta/a/datos/productos_cambios.csv'
INTO TABLE stg_producto_cambio
FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n' IGNORE 1 ROWS;

-- Repita el mismo patron para:
-- tiendas.csv -> stg_tienda
-- tiendas_cambios.csv -> stg_tienda_cambio
-- clientes.csv -> stg_cliente
-- clientes_cambios.csv -> stg_cliente_cambio
-- promociones.csv -> stg_promocion
-- ventas.csv -> stg_venta_pos
```

Figura 9. Carga de los archivos al área de staging.

Si los archivos fueron creados en Windows

Algunos editores guardan los saltos de línea como retorno de carro más avance de línea. Si observa que el último campo de cada fila arrastra un carácter extraño o que los conteos no cuadran, cambie la cláusula por LINES TERMINATED BY '\r\n'. Compruébelo con SELECT * FROM stg_producto LIMIT 3 antes de continuar: un error aquí se propaga a todo el laboratorio.

Verifique inmediatamente los conteos. Si alguno no coincide con la tabla 2, deténgase y corrija antes de seguir:

```
SELECT 'stg_producto' AS tabla, COUNT(*) AS filas FROM stg_producto
UNION ALL SELECT 'stg_producto_cambio', COUNT(*) FROM stg_producto_cambio
UNION ALL SELECT 'stg_tienda', COUNT(*) FROM stg_tienda
UNION ALL SELECT 'stg_tienda_cambio', COUNT(*) FROM stg_tienda_cambio
UNION ALL SELECT 'stg_cliente', COUNT(*) FROM stg_cliente
UNION ALL SELECT 'stg_cliente_cambio', COUNT(*) FROM stg_cliente_cambio
UNION ALL SELECT 'stg_promoción', COUNT(*) FROM stg_promoción
UNION ALL SELECT 'stg_venta_pos', COUNT(*) FROM stg_venta_pos;
```

Figura 10. Verificación de la carga de staging.

Tabla	Filas esperadas	Tabla	Filas esperadas
stg_producto	386	stg_cliente	1 200
stg_producto_cambio	28	stg_cliente_cambio	95
stg_tienda	34	stg_promocion	12
stg_tienda_cambio	6	stg_venta_pos	106 708

Tabla 4. Conteos esperados tras la carga de staging.

6. Etapa 4: Dimensión tiempo y filas especiales

6.1. Generación de la dimensión tiempo

La dimensión tiempo no se extrae: se genera. Se construye para un rango más amplio que el de los datos, de modo que no haya que regenerarla en cada carga. Aquí se genera del 1 de enero de 2024 al 31 de diciembre de 2027, es decir, 1 461 filas.

La técnica consiste en producir una secuencia de números con una expresión de tabla común recursiva y sumarlos a la fecha inicial:

```
SET SESSION cte_max_recursion_depth = 5000;

INSERT INTO dim_tiempo
(tiempo_key, fecha, anio, trimestre, mes, nombre_mes, anio_mes,
 dia_mes, dia_semana, nombre_dia, es_fin_semana, es_dia_pago)
WITH RECURSIVE seq AS (
  SELECT DATE('2024-01-01') AS f
  UNION ALL
  SELECT DATE_ADD(f, INTERVAL 1 DAY) FROM seq WHERE f < '2027-12-31'
)
SELECT
  CAST(DATE_FORMAT(f, '%Y%m%d') AS UNSIGNED),
  f,
  YEAR(f),
  QUARTER(f),
  MONTH(f),
  ELT(MONTH(f), 'Enero','Febrero','Marzo','Abril','Mayo','Junio',
           'Julio','Agosto','Setiembre','Octubre','Noviembre','Diciembre'),
  DATE_FORMAT(f, '%Y-%m'),
  DAYOFMONTH(f),
  WEEKDAY(f) + 1,
  ELT(WEEKDAY(f) + 1, 'Lunes','Martes','Miercoles','Jueves','Viernes',
           'Sabado','Domingo'),
  CASE WHEN WEEKDAY(f) >= 5 THEN 1 ELSE 0 END,
  CASE WHEN DAYOFMONTH(f) IN (15, 16) OR f = LAST_DAY(f) THEN 1 ELSE 0 END
FROM seq;

SELECT COUNT(*) FROM dim_tiempo; -- debe devolver 1461
```

Figura 11. Generación de la dimensión tiempo.

Equivalencias en otros motores

PostgreSQL: INSERT INTO dim_tiempo SELECT ... FROM generate_series('2024-01-01'::date, '2027-12-31'::date, '1 day') AS f.

SQL Server: use una CTE recursiva con OPTION (MAXRECURSION 0), o una tabla de números.

En cualquier motor, el atributo es_dia_pago es una regla de negocio de MERCANDINA (quincena y fin de mes), no una función del calendario: es precisamente el tipo de atributo que justifica tener una dimensión tiempo propia en lugar de usar las funciones de fecha del motor.

6.2. Filas especiales

Antes de cargar un solo dato real, inserte las filas de valores desconocidos y no aplicables. El autoincremento no permite valores negativos automáticos, así que se insertan de forma explícita.

```
-- Producto y tienda: -1 = Desconocido
INSERT INTO dim_producto
  (producto_key, producto_id, nombre_producto, marca, categoria,
   subcategoria, unidad_medida, es_marca_propia, fecha_inicio_vig,
   fecha_fin_vig, es_vigente)
VALUES (-1, 'DESCONOCIDO', 'Desconocido', 'Desconocido', 'Desconocido',
        'Desconocido', 'NA', 0, '1900-01-01', '9999-12-31', 1);

INSERT INTO dim_tienda
  (tienda_key, tienda_id, nombre_tienda, formato, distrito, ciudad,
   region, area_m2, fecha_inicio_vig, fecha_fin_vig, es_vigente)
VALUES (-1, 'DESCONOCIDO', 'Desconocido', 'Desconocido', 'Desconocido',
        'Desconocido', 'Desconocido', 0, '1900-01-01', '9999-12-31', 1);

-- Cliente: -1 = Desconocido, -2 = No aplica (venta anonima)
INSERT INTO dim_cliente
  (cliente_key, cliente_id, nombre, segmento, distrito, ciudad,
   fecha_inicio_vig, fecha_fin_vig, es_vigente)
VALUES
  (-1, 'DESCONOCIDO', 'Desconocido', 'Desconocido', 'Desconocido',
   'Desconocido', '1900-01-01', '9999-12-31', 1),
  (-2, 'NO APLICA', 'Venta anonima', 'No aplica', 'No aplica',
   'No aplica', '1900-01-01', '9999-12-31', 1);

-- Promocion: -2 = No aplica (la mayoría de las ventas no lleva promocion)
INSERT INTO dim_promocion
  (promocion_key, promocion_id, nombre_promocion, tipo_promocion,
   fecha_inicio, fecha_fin)
VALUES (-2, 'NO APLICA', 'Sin promocion', 'No aplica',
        '1900-01-01', '9999-12-31');
```

Figura 12. Inserción de las filas especiales.

La distinción entre -1 y -2 no es cosmética y debe quedar justificada en su informe: -1 significa «el dato existía y no lo encontramos», que es un problema de calidad a investigar; -2 significa «el dato no corresponde», que es una situación legítima del negocio. Una venta anónima no es un error: es una venta sin cliente fidelizado, y confundir ambos casos haría que el indicador de calidad de la etapa 7 informara un 55 % de problemas donde no hay ninguno.

7. Etapa 5: Carga inicial de las dimensiones

Continúe en dm_mercandina_dim.sql. La carga inicial establece la vigencia de todas las filas desde 1900-01-01, de modo que cualquier hecho anterior al primer cambio encuentre siempre una versión válida.

```

INSERT INTO dim_producto
(producto_id, nombre_producto, marca, categoria, subcategoria,
unidad_medida, es_marca_propia, precio_lista, costo_estandar,
fecha_inicio_vig, fecha_fin_vig, es_vigente)
SELECT cod_producto, nombre_producto, marca, categoria, subcategoria,
unidad_medida, CAST(es_marca_propia AS UNSIGNED),
CAST(precio_lista AS DECIMAL(12,2)),
CAST(costo_unitario AS DECIMAL(12,2)),
'1900-01-01', '9999-12-31', 1
FROM stg_producto;

INSERT INTO dim_tienda
(tienda_id, nombre_tienda, formato, distrito, ciudad, region,
area_m2, fecha_apertura, fecha_inicio_vig, fecha_fin_vig, es_vigente)
SELECT cod_tienda, nombre_tienda, formato, distrito, ciudad, region,
CAST(area_m2 AS DECIMAL(10,2)), STR_TO_DATE(fecha_apertura, '%Y-%m-%d'),
'1900-01-01', '9999-12-31', 1
FROM stg_tienda;

INSERT INTO dim_cliente
(cliente_id, nombre, segmento, distrito, ciudad, fecha_alta,
fecha_inicio_vig, fecha_fin_vig, es_vigente)
SELECT doc_cliente, nombre, segmento, distrito, ciudad,
STR_TO_DATE(fecha_alta, '%Y-%m-%d'),
'1900-01-01', '9999-12-31', 1
FROM stg_cliente;

INSERT INTO dim_promocion
(promocion_id, nombre_promocion, tipo_promocion, fecha_inicio, fecha_fin)
SELECT cod_promocion, nombre_promocion, tipo_promocion,
STR_TO_DATE(fecha_inicio, '%Y-%m-%d'),
STR_TO_DATE(fecha_fin, '%Y-%m-%d')
FROM stg_promocion;

-- La dimension junk se construye con las combinaciones REALMENTE observadas
INSERT INTO dim_transaccion (tipo_comprobante, forma_pago, canal, es_devolucion)
SELECT DISTINCT tipo_comprobante, forma_pago, canal,
CAST(flag_devolucion AS UNSIGNED)
FROM stg_venta_pos;
    
```

Figura 13. Carga inicial de las dimensiones.

Verifique los conteos antes de continuar. Recuerde sumar las filas especiales:

Dimensión	Filas esperadas	Composición
dim_tiempo	1 461	4 años completos de calendario
dim_producto	387	386 del catálogo + 1 fila «Desconocido»
dim_tienda	35	34 locales + 1 fila «Desconocido»
dim_cliente	1 202	1 200 clientes + «Desconocido» + «No aplica»
dim_promocion	13	12 campañas + 1 fila «No aplica»
dim_transaccion	29	Combinaciones observadas en las ventas

Tabla 5. Conteos esperados tras la carga inicial de dimensiones.

Una observación que debe explicar en su informe

dim_transaccion tiene 29 filas, no 32. El producto cartesiano de 2 tipos de comprobante, 4 formas de pago, 2 canales y 2 valores del indicador de devolución daría 32 combinaciones, pero solo 29 aparecen realmente en los datos del semestre. Explique en su informe qué riesgo introduce construir la dimensión junk con DISTINCT sobre los datos observados en lugar del producto cartesiano completo, y en qué momento se manifestaría ese riesgo.

8. Etapa 6: Aplicación de los cambios SCD Tipo 2

Cree el archivo `dm_mercandina_scd2.sql`. El 1 de abril de 2026 la empresa aplicó tres cambios simultáneos: reclasificó 28 productos de categoría, convirtió 6 tiendas del formato Express al formato Market ampliando su área, y actualizó el distrito y el segmento de 95 clientes que notificaron una mudanza.

El patrón SCD Tipo 2 tiene exactamente dos pasos, y el orden importa: primero se cierra la versión vigente y después se inserta la nueva. Si se invierte el orden, el paso de cierre alcanzaría también a la fila recién insertada y la dimensión quedaría sin ninguna versión vigente.

```
-- ===== PASO 1: cerrar la version vigente =====
UPDATE dim_producto d
JOIN   stg_producto_cambio ch ON ch.cod_producto = d.producto_id
SET    d.fecha_fin_vig = DATE_SUB(STR_TO_DATE(ch.fecha_cambio, '%Y-%m-%d'),
                                INTERVAL 1 DAY),
      d.es_vigente     = 0
WHERE  d.es_vigente = 1;

-- ===== PASO 2: insertar la nueva version =====
INSERT INTO dim_producto
(producto_id, nombre_producto, marca, categoria, subcategoria,
unidad_medida, es_marca_propia, precio_lista, costo_estandar,
fecha_inicio_vig, fecha_fin_vig, es_vigente)
SELECT ch.cod_producto, ch.nombre_producto, ch.marca, ch.categoria,
ch.subcategoria, ch.unidad_medida,
CAST(ch.es_marca_propia AS UNSIGNED),
CAST(ch.precio_lista AS DECIMAL(12,2)),
CAST(ch.costo_unitario AS DECIMAL(12,2)),
STR_TO_DATE(ch.fecha_cambio, '%Y-%m-%d'), '9999-12-31', 1
FROM   stg_producto_cambio ch;
```

Figura 14. Patrón SCD Tipo 2 aplicado a la dimensión producto.

Repita el mismo patrón de dos pasos para `dim_tienda` (usando `stg_tienda_cambio`) y para `dim_cliente` (usando `stg_cliente_cambio`), adaptando las columnas. Al terminar, la situación de las dimensiones debe ser:

Dimensión	Filas totales	Filas vigentes	Filas cerradas
<code>dim_producto</code>	415	387	28
<code>dim_tienda</code>	41	35	6
<code>dim_cliente</code>	1 297	1 202	95

Tabla 6. Estado de las dimensiones tras aplicar el SCD Tipo 2.

Compruebe el resultado sobre un producto concreto. Debe ver dos filas: una con vigencia cerrada el 31 de marzo y otra abierta desde el 1 de abril, con categorías distintas y claves subrogadas distintas.

```
SELECT producto_key, producto_id, categoria, subcategoria,
       fecha_inicio_vig, fecha_fin_vig, es_vigente
FROM   dim_producto
WHERE  producto_id = (SELECT cod_producto FROM stg_producto_cambio LIMIT 1)
ORDER BY fecha_inicio_vig;
```

Figura 15. Verificación del historial de un producto reclasificado.

El error que arruina el laboratorio en este punto

Es habitual escribir el paso 1 sin la condición `WHERE d.es_vigente = 1`, pensando que la unión con la tabla de cambios ya limita las filas afectadas. No es así: sin esa condición, el `UPDATE` volvería a tocar las versiones ya cerradas y les asignaría de nuevo la misma fecha de cierre, ocultando cualquier historia previa que existiera.

Advertencia complementaria: este script NO es idempotente. Ejecutarlo dos veces insertaría 28 filas adicionales y dejaría la dimensión con dos versiones vigentes por producto, lo que hará fallar el control 6 de la etapa 7. Si necesita repetir la carga, vuelva a ejecutar los tres scripts desde el principio sobre la base de datos recreada. Convertirlo en idempotente —comparando el hash de los atributos antes de insertar— es precisamente una de las opciones de la actividad propuesta.

9. Etapa 7: Carga de la tabla de hechos

Cree el archivo `dm_mercandina_carga.sql`. Esta es la sentencia central del laboratorio: sustituye las claves naturales del sistema fuente por las claves subrogadas del datamart, recuperando en cada caso la versión de la dimensión que estaba vigente en la fecha de la transacción.

```
USE dm_mercandina;

INSERT INTO fact_venta
  (tiempo_key, producto_key, tienda_key, cliente_key, promocion_key,
   transaccion_key, nro_ticket, cantidad, importe_venta, importe_costo,
   importe_descuento)
SELECT
  t.tiempo_key,
  COALESCE(p.producto_key, -1),      -- producto no hallado en el catalogo
  COALESCE(s.tienda_key, -1),
  COALESCE(cl.cliente_key, -2),      -- venta anonima (no es un error)
  COALESCE(pr.promocion_key, -2),    -- venta sin promocion
  tr.transaccion_key,
  o.nro_ticket,
  CAST(o.cantidad AS DECIMAL(12,3)),
  CAST(o.importe_neto AS DECIMAL(18,2)),
  CAST(o.costo_unitario AS DECIMAL(18,2))
  * CAST(o.cantidad AS DECIMAL(12,3)),
  CAST(o.descuento AS DECIMAL(18,2))
FROM
  stg_venta_pos o
JOIN
  dim_tiempo t
  ON t.fecha = STR_TO_DATE(o.fecha_emision, '%Y-%m-%d')
LEFT JOIN
  dim_producto p
  ON p.producto_id = o.cod_producto
  AND STR_TO_DATE(o.fecha_emision, '%Y-%m-%d')
  BETWEEN p.fecha_inicio_vig AND p.fecha_fin_vig
LEFT JOIN
  dim_tienda s
  ON s.tienda_id = o.cod_tienda
  AND STR_TO_DATE(o.fecha_emision, '%Y-%m-%d')
  BETWEEN s.fecha_inicio_vig AND s.fecha_fin_vig
LEFT JOIN
  dim_cliente cl
  ON cl.cliente_id = NULLIF(o.doc_cliente, '')
  AND STR_TO_DATE(o.fecha_emision, '%Y-%m-%d')
  BETWEEN cl.fecha_inicio_vig AND cl.fecha_fin_vig
LEFT JOIN
  dim_promocion pr
  ON pr.promocion_id = NULLIF(o.cod_promocion, '')
JOIN
  dim_transaccion tr
  ON tr.tipo_comprobante = o.tipo_comprobante
  AND tr.forma_pago = o.forma_pago
  AND tr.canal = o.canal
  AND tr.es_devolucion = CAST(o.flag_devolucion AS UNSIGNED);
```

Figura 16. Carga de hechos con búsqueda de claves subrogadas sensible a la fecha.

Cinco detalles de esta sentencia determinan si la carga es correcta o silenciosamente falsa. Anótelos en su informe:

Detalle	Por qué es indispensable
LEFT JOIN hacia las dimensiones, no INNER JOIN	Un INNER JOIN descartaría las 362 líneas cuyo producto no existe en el catálogo. El total del datamart dejaría de coincidir con el del origen y la diferencia sería invisible.
COALESCE hacia la fila especial	Convierte la ausencia en un valor explícito y trazable. Sin él quedarían NULL en las claves foráneas, que rompen los joins de consulta y dan totales distintos según el orden de las tablas.

BETWEEN sobre las fechas de vigencia	Es lo que da sentido al SCD Tipo 2. Si en su lugar se filtrara por es_vigente = 1, toda la venta de enero a marzo quedaría atribuida a la categoría que el producto tiene desde abril.
NULLIF sobre los campos vacíos	En el CSV, la ausencia de cliente o de promoción es una cadena vacía, no un NULL. Sin NULLIF, el join intentaría emparejar la cadena vacía y el COALESCE nunca se activaría.
JOIN normal hacia dim_transaccion	Aquí sí es correcto: la dimensión junk se construyó con las combinaciones observadas, de modo que toda fila del origen tiene por fuerza su correspondencia.

Tabla 7. Decisiones críticas de la sentencia de carga.

Si la carga tarda demasiado

Con los índices de staging creados en la etapa 2, la sentencia debería completarse en menos de dos minutos en un equipo corriente. Si tarda mucho más, verifique que creó los índices ix_stg_fecha e ix_stg_prod, y que las dimensiones tienen sus índices sobre la clave natural. La conversión repetida STR_TO_DATE sobre cada fila es costosa: una optimización válida, que puede aplicar y comentar en su informe, es añadir a stg_venta_pos una columna DATE calculada y poblarla una sola vez antes de la carga.

10. Etapa 8: Cuadros de control

La carga no termina cuando el motor deja de trabajar sin error: termina cuando se ha demostrado que el resultado es correcto. Ejecute los siete controles siguientes, capture el resultado de cada uno y escriba una línea interpretándolo. El conjunto constituye el archivo cuadros_control.pdf.

10.1. Controles 1 y 2: conteo e importe

```
-- Control 1: ninguna fila del origen se perdió ni se duplico
SELECT (SELECT COUNT(*) FROM stg_venta_pos) AS filas_origen,
       (SELECT COUNT(*) FROM fact_venta)    AS filas_hecho,
       (SELECT COUNT(*) FROM stg_venta_pos)
       - (SELECT COUNT(*) FROM fact_venta) AS diferencia;

-- Control 2: el dinero cuadra al centimo
SELECT ROUND((SELECT SUM(CAST(importe_netto AS DECIMAL(18,2)))
              FROM stg_venta_pos), 2)          AS importe_origen,
       ROUND((SELECT SUM(importe_venta)
              FROM fact_venta), 2)              AS importe_hecho;
```

Figura 17. Controles de conteo y de importe.

10.2. Controles 3 y 4: integridad y calidad

```
-- Control 3: ninguna clave foranea sin correspondencia
SELECT COUNT(*) AS huérfanos_producto
FROM fact_venta f
LEFT JOIN dim_producto p ON p.producto_key = f.producto_key
WHERE p.producto_key IS NULL; -- repita para tienda, cliente y promoción

-- Control 4: cuanta calidad de dato estamos absorbiendo
SELECT ROUND(100.0 * SUM(producto_key = -1) / COUNT(*), 3) AS pct_prod_desc,
       ROUND(100.0 * SUM(tienda_key   = -1) / COUNT(*), 3) AS pct_tienda_desc,
       ROUND(100.0 * SUM(cliente_key  = -2) / COUNT(*), 2) AS pct_anonimas,
       ROUND(100.0 * SUM(promoción_key = -2) / COUNT(*), 2) AS pct_sin_promo
FROM fact_venta;
```

Figura 18. Controles de integridad y de calidad.

10.3. Controles 5, 6 y 7: grano, vigencia y cobertura

```
-- Control 5: el grano declarado no admite duplicados
SELECT tiempo_key, producto_key, tienda_key, nro_ticket, COUNT(*) AS veces
FROM fact_venta
GROUP BY tiempo_key, producto_key, tienda_key, nro_ticket
HAVING COUNT(*) > 1;

-- Control 6: ninguna clave natural puede tener dos versiones vigentes
SELECT 'producto' AS dimension, producto_id AS clave, COUNT(*) AS vigentes
FROM dim_producto WHERE es_vigente = 1 GROUP BY producto_id
HAVING COUNT(*) > 1
UNION ALL
SELECT 'tienda', tienda_id, COUNT(*)
FROM dim_tienda WHERE es_vigente = 1 GROUP BY tienda_id HAVING COUNT(*) > 1
UNION ALL
SELECT 'cliente', cliente_id, COUNT(*)
FROM dim_cliente WHERE es_vigente = 1 GROUP BY cliente_id HAVING COUNT(*) > 1;

-- Control 7: toda fecha del origen existe en la dimension tiempo
SELECT COUNT(*) AS fechas_sin_dimension
FROM stg_venta_pos o
LEFT JOIN dim_tiempo t
ON t.fecha = STR_TO_DATE(o.fecha_emision, '%Y-%m-%d')
WHERE t.tiempo_key IS NULL;
```

Figura 19. Controles de grano, vigencia y cobertura temporal.

10.4. Resultados esperados

Compare sus resultados con la tabla siguiente. Si alguno no coincide, no continúe: vuelva a la etapa correspondiente y corrija. Un datamart que no cuadra no sirve de nada, por bien diseñado que esté.

N.º	Control	Resultado esperado	Si no coincide, revise
1	Conteo de filas	106 708 = 106 708, diferencia 0	La carga de staging y el JOIN con dim_tiempo
2	Cuadre de importes	3 430 651.54 en ambos	El tipo DECIMAL y la conversión del importe
3	Huérfanos	0 en las cuatro dimensiones	Las filas especiales y los COALESCE
4	Calidad	0.339 % producto -1; 54.54 % anónimas; 77.86 % sin promoción	El uso de NULLIF y la distinción entre -1 y -2
5	Duplicados de grano	0 filas	Que no haya ejecutado dos veces la carga de hechos
6	Versiones vigentes	0 filas	La condición WHERE es_vigente = 1 del paso 1 del SCD
7	Cobertura temporal	0 filas	El rango de la dimensión tiempo

Tabla 8. Resultados esperados de los siete cuadros de control.

Interpretación del control 4: no todo porcentaje alto es un problema

El 54,54 % de ventas anónimas y el 77,86 % de ventas sin promoción son cifras esperadas del negocio, no defectos: la mayoría de las compras en una tienda de conveniencia no involucra al programa de fidelización ni a una campaña. Por eso apuntan a la clave -2, «No aplica».

El 0,339 % de productos desconocidos sí es un problema de calidad: son siete códigos que el POS registró y que el ERP no reconoce. En un proyecto real, este número se reporta al área responsable del catálogo y se fija un umbral de tolerancia. Indique en su informe qué umbral propondría para MERCANDINA y qué haría si se superara.

11. Etapa 9: Operaciones OLAP y materialización del cubo

Cree el archivo `dm_mercandina_olap.sql`. Escriba una consulta por cada operación, con un comentario que indique cuál es. Los resultados de referencia le permiten verificar que su carga es correcta.

11.1. Las seis operaciones

```
-- (1) ROLL-UP: del detalle mensual al total trimestral por ciudad
SELECT t.anio, t.trimestre, s.ciudad, SUM(f.importe_venta) AS venta
FROM fact_venta f
JOIN dim_tiempo t ON t.tiempo_key = f.tiempo_key
JOIN dim_tienda s ON s.tienda_key = f.tienda_key
GROUP BY t.anio, t.trimestre, s.ciudad
ORDER BY 1, 2, 4 DESC;

-- (2) DRILL-DOWN: de categoria a subcategoria dentro de un mes
SELECT p.categoria, p.subcategoria, SUM(f.importe_venta) AS venta
FROM fact_venta f
JOIN dim_tiempo t ON t.tiempo_key = f.tiempo_key
JOIN dim_producto p ON p.producto_key = f.producto_key
WHERE t.anio_mes = '2026-05'
GROUP BY p.categoria, p.subcategoria
ORDER BY p.categoria, venta DESC;

-- (3) SLICE: se fija una sola dimension
SELECT t.anio_mes, SUM(f.importe_venta) AS venta
FROM fact_venta f
JOIN dim_tiempo t ON t.tiempo_key = f.tiempo_key
JOIN dim_tienda s ON s.tienda_key = f.tienda_key
WHERE s.ciudad = 'Arequipa'
GROUP BY t.anio_mes ORDER BY 1;

-- (4) DICE: se restringen varias dimensiones a la vez
SELECT s.ciudad, p.categoria, SUM(f.importe_venta) AS venta
FROM fact_venta f
JOIN dim_tiempo t ON t.tiempo_key = f.tiempo_key
JOIN dim_tienda s ON s.tienda_key = f.tienda_key
JOIN dim_producto p ON p.producto_key = f.producto_key
WHERE s.ciudad IN ('Lima', 'Cusco')
AND p.categoria IN ('Bebidas', 'Snacks', 'Lacteos')
AND t.anio_mes BETWEEN '2026-04' AND '2026-06'
GROUP BY s.ciudad, p.categoria;

-- (5) PIVOT: los meses pasan de filas a columnas
SELECT s.ciudad,
SUM(CASE WHEN t.anio_mes = '2026-01' THEN f.importe_venta END) AS ene,
SUM(CASE WHEN t.anio_mes = '2026-02' THEN f.importe_venta END) AS feb,
SUM(CASE WHEN t.anio_mes = '2026-03' THEN f.importe_venta END) AS mar,
SUM(CASE WHEN t.anio_mes = '2026-04' THEN f.importe_venta END) AS abr,
SUM(CASE WHEN t.anio_mes = '2026-05' THEN f.importe_venta END) AS may,
SUM(CASE WHEN t.anio_mes = '2026-06' THEN f.importe_venta END) AS jun
FROM fact_venta f
JOIN dim_tiempo t ON t.tiempo_key = f.tiempo_key
JOIN dim_tienda s ON s.tienda_key = f.tienda_key
GROUP BY s.ciudad;

-- (6) DRILL-ACROSS: en la etapa 9 se resuelve con una sola tabla de hechos,
-- comparando el periodo con y sin promocion sobre la misma dimension
WITH con_promo AS (
SELECT p.categoria, SUM(f.importe_venta) AS venta_promo
FROM fact_venta f JOIN dim_producto p ON p.producto_key = f.producto_key
WHERE f.promocion_key <> -2 GROUP BY p.categoria
), sin_promo AS (
SELECT p.categoria, SUM(f.importe_venta) AS venta_normal
FROM fact_venta f JOIN dim_producto p ON p.producto_key = f.producto_key
WHERE f.promocion_key = -2 GROUP BY p.categoria
)
SELECT COALESCE(a.categoria, b.categoria) AS categoria,
a.venta_promo, b.venta_normal,
ROUND(100.0 * a.venta_promo /
NULLIF(a.venta_promo + b.venta_normal, 0), 2) AS pct_promocionado
```

```
FROM con_promo a LEFT JOIN sin_promo b ON a.categoria = b.categoria;
```

Figura 20. Las seis operaciones OLAP en SQL.

11.2. Materialización del cubo con GROUPING SETS

MySQL 8 no implementa GROUPING SETS ni CUBE, pero sí WITH ROLLUP y la función GROUPING. Escriba la versión de ROLLUP y, además, la de GROUPING SETS que ejecutaría en un motor que la soporte, dejándola comentada en el archivo.

```
-- MySQL 8: agregados jerarquicos con WITH ROLLUP
SELECT COALESCE(t.anio_mes, '== TOTAL ==') AS periodo,
       COALESCE(s.ciudad, '-- todas --') AS ciudad,
       ROUND(SUM(f.importe_venta), 2) AS venta,
       GROUPING(s.ciudad) AS es_subtotal_periodo,
       GROUPING(t.anio_mes) AS es_total_general
FROM fact_venta f
JOIN dim_tiempo t ON t.tiempo_key = f.tiempo_key
JOIN dim_tienda s ON s.tienda_key = f.tienda_key
GROUP BY t.anio_mes, s.ciudad WITH ROLLUP;

-- Equivalente en PostgreSQL, SQL Server u Oracle (dejar comentado):
-- GROUP BY GROUPING SETS (
-- (t.anio_mes, s.ciudad), -- venta mensual por ciudad
-- (t.anio_mes),          -- venta mensual total
-- (s.ciudad),            -- venta del semestre por ciudad
-- () );                  -- total general
```

Figura 21. Materialización de un cubo en SQL.

Explique en su informe la diferencia entre ambas construcciones: ROLLUP produce solo los niveles jerárquicos descendentes, mientras que GROUPING SETS permite elegir exactamente qué combinaciones calcular, incluida la de ciudad sin periodo, que ROLLUP no genera.

11.3. Valores de referencia

Los resultados siguientes deben coincidir con los suyos. Si difieren, hay un problema en alguna etapa anterior.

Indicador	Valor esperado
Venta total del semestre	3 430 651.54
Tickets distintos	32 613
Ticket promedio	105.19
Margen porcentual global	26.76 %
Venta de Lima	1 780 310.66
Venta de Arequipa	621 510.02
Venta de Cusco	521 301.04
Venta de Trujillo	507 529.82
Mes de mayor venta	Mayo de 2026, con 619 775.27
Mes de menor venta	Febrero de 2026, con 528 200.10

Tabla 9. Valores de referencia para verificar la carga.

El margen se calcula, no se almacena

Para obtener el 26,76 % debe escribir $(\text{SUM}(\text{importe_venta}) - \text{SUM}(\text{importe_costo})) / \text{SUM}(\text{importe_venta})$. Si en su lugar calculara el margen por fila y luego promediara esos porcentajes, obtendría un número distinto y equivocado. Compruébelo: ejecute las dos versiones y muestre ambos resultados en su informe, explicando cuál es correcto y por qué.

12. Etapa 10: Modelo semántico en Power BI

Abra Power BI Desktop y siga los pasos en orden. El objetivo no es producir un informe bonito, sino un modelo correcto: la evaluación se centra en las relaciones, las medidas y el ocultamiento de las columnas técnicas.

Paso	Acción	Criterio de verificación
1	Obtener datos > Base de datos MySQL. Servidor: localhost. Base de datos: dm_mercandina.	Si Power BI pide el conector MySQL, instale el paquete «MySQL Connector/NET» y reinicie.
2	Seleccionar únicamente fact_venta y las seis dimensiones. NO importar las tablas stg_.	El navegador debe mostrar siete tablas seleccionadas, ninguna de staging.
3	Cargar en modo Importación.	La carga de 106 708 filas debe tardar menos de un minuto.
4	Abrir la vista de Modelo y revisar el diagrama.	Las seis dimensiones rodean a fact_venta. No debe existir ninguna relación entre dos dimensiones.
5	Verificar cada relación: cardinalidad uno a varios y dirección de filtro simple, desde la dimensión hacia el hecho.	Corrija cualquier relación que Power BI haya detectado como bidireccional o de varios a varios.
6	Seleccionar dim_tiempo y marcarla como tabla de fechas, usando la columna fecha.	Sin este paso, las funciones de inteligencia de tiempo dan resultados incorrectos.
7	Ocultar todas las columnas de clave (los campos terminados en _key) y las columnas técnicas de vigencia.	En la vista de informe no debe verse ninguna clave subrogada.
8	Crear una tabla vacía llamada _Medidas y alojar en ella todas las medidas.	Agrupar las medidas y evita que queden dispersas entre las tablas.
9	Construir las jerarquías: Región > Ciudad > Distrito > Tienda, y Categoría > Subcategoría > Producto.	Deben aparecer como jerarquías navegables en el panel de campos.
10	Validar el total contra el cuadro de la etapa 8.	La medida Importe Venta debe dar 3 430 651.54.

Tabla 10. Construcción del modelo semántico.

12.1. Medidas obligatorias

Cree estas ocho medidas en DAX. Ninguna de ellas puede implementarse como columna calculada:

Importe Venta	= SUM (fact_venta[importe_venta])
Importe Costo	= SUM (fact_venta[importe_costo])
Unidades	= SUM (fact_venta[cantidad])
Tickets	= DISTINCTCOUNT (fact_venta[nro_ticket])
Margen	= [Importe Venta] - [Importe Costo]
Margen %	= DIVIDE ([Margen], [Importe Venta])
Ticket Promedio	= DIVIDE ([Importe Venta], [Tickets])
Venta por m2	= DIVIDE ([Importe Venta], SUM (dim_tienda[area_m2]))

Figura 22. Medidas del modelo semántico.

12.2. Página de operaciones OLAP

En una única página del informe, construya:

- Una matriz con la jerarquía de producto en filas y la de tiempo en columnas, con la medida Importe Venta. Debe permitir expandir y contraer niveles: eso demuestra roll-up y drill-down.
- Dos segmentadores, uno de ciudad y otro de categoría, que demuestren slice y dice.
- Una tarjeta con el total de Importe Venta, para verificar visualmente el cuadro.

- Una segunda matriz con Margen % y Ticket Promedio por ciudad, que demuestre que los ratios se comportan correctamente al agregar.

Prueba final del modelo

Arrastre la columna producto_key a una tarjeta y observe qué ocurre: Power BI la suma y produce un número sin ningún significado. Ese es el motivo exacto por el que el paso 7 exige ocultarla. Tome la captura, inclúyala en su informe y explique el riesgo en dos líneas. Después vuelva a ocultar la columna.

13. Actividad propuesta (40 minutos)

Esta actividad es individual y se desarrolla al terminar la guía dirigida. Se entrega como actividad_propuesta.pdf, junto con las sentencias SQL que haya escrito, incorporadas al archivo dm_mercandina_olap.sql bajo el encabezado «Actividad propuesta».

Situación planteada

La Gerencia Comercial de MERCANDINA ha recibido el datamart que usted acaba de construir y plantea una objeción y una necesidad nueva.

La objeción: «El informe de ventas por categoría de marzo cambió respecto del que nos entregaron en abril. Antes Snacks aparecía con más venta y ahora aparece con menos. ¿Los datos están mal?».

La necesidad: la Gerencia quiere medir el desempeño de las promociones, pero no con la venta promocionada —que ya tienen— sino con la efectividad: qué proporción de la venta potencial de cada categoría fue captada por una campaña, y si las categorías que más se promocionan son las que mejor margen dejan.

13.1. Tareas

N.º	Tarea	Tiempo	Producto
1	Redacte la respuesta a la objeción de la Gerencia, en un máximo de 200 palabras y en lenguaje de negocio, sin jerga técnica. Debe explicar por qué el informe cambió, si eso es un error o un comportamiento correcto, y qué decisión de diseño lo produjo.	10 min	Párrafo en el PDF
2	Demuéstrelo con datos: escriba una consulta que muestre, para los 28 productos reclasificados, la venta atribuida a su categoría antigua y a la nueva, con el corte del 1 de abril de 2026.	10 min	Consulta y resultado
3	Construya el indicador de efectividad promocional por categoría: venta con promoción, venta sin promoción, porcentaje promocionado, descuento total otorgado y margen porcentual resultante.	12 min	Consulta y resultado
4	Analice el resultado y responda en un máximo de 150 palabras: ¿las categorías más promocionadas son las de mejor margen? ¿Qué recomendaría a la Gerencia y qué dato le falta para estar seguro?	8 min	Párrafo en el PDF

Tabla 11. Tareas de la actividad propuesta.

13.2. Orientaciones

Para la tarea 2, la clave está en unir la tabla de hechos con la dimensión producto y agrupar por categoría, aprovechando que un mismo producto_id tiene dos claves subrogadas distintas con categorías distintas. La consulta debe partir de la lista de productos que cambiaron:

```
-- Punto de partida sugerido para la tarea 2
SELECT p.producto_id,
       p.categoria,
       p.fecha_inicio_vig,
       p.fecha_fin_vig,
       COUNT(*) AS lineas,
       ROUND(SUM(f.importe_venta), 2) AS venta
FROM   fact_venta f
JOIN   dim_producto p ON p.producto_key = f.producto_key
WHERE  p.producto_id IN (SELECT cod_producto FROM stg_producto_cambio)
GROUP BY p.producto_id, p.categoria, p.fecha_inicio_vig, p.fecha_fin_vig
ORDER BY p.producto_id, p.fecha_inicio_vig;
```

Figura 23. Punto de partida para demostrar el efecto del SCD Tipo 2.

Para la tarea 3, recuerde el principio de aditividad: el porcentaje promocionado y el margen no se promedian, se calculan a partir de sumas. La estructura será una consulta con agregación condicional sobre `promocion_key`, distinguiendo el valor -2 del resto. Tenga cuidado con el signo del descuento en las líneas de devolución.

Criterio de calidad de la actividad

Lo que se evalúa no es la complejidad del SQL sino la calidad del razonamiento. Una respuesta que diga «los datos están bien porque usamos SCD Tipo 2» no vale: la Gerencia no sabe qué es eso. Una respuesta que explique que el informe de marzo se emitió antes de la reclasificación y que el datamart conserva deliberadamente la categoría que cada producto tenía en el momento de la venta, de modo que ambos informes son correctos en su propio contexto, sí vale. Y debe indicar cuál de los dos usar para comparar la evolución de una categoría a lo largo del año, que es la pregunta que la Gerencia realmente tiene.

13.3. Extensión optativa

Si termina antes del tiempo previsto, puede desarrollar una de estas dos extensiones. No suma puntos a la rúbrica, pero se considera en la evaluación continua:

- Convierta el script de SCD Tipo 2 en idempotente: añada a las dimensiones una columna `hash_atributos` con el SHA-256 de los atributos historizados, y modifique la carga para que solo cierre e inserte cuando el hash haya cambiado. Verifique que ejecutarlo dos veces deja la dimensión igual.
- Construya una tabla agregada `agg_venta_dia_categoria` con la venta diaria por categoría y tienda, mida el tiempo de la consulta de ventas mensuales por categoría contra la tabla atómica y contra el agregado, y documente la diferencia junto con el costo de mantenimiento que introduce.

14. Rúbrica de evaluación

Criterio	Pts	Nivel esperado
Diseño lógico y DDL	25	Claves subrogadas y naturales correctas; filas especiales presentes con la distinción entre -1 y -2 justificada; clave primaria igual al grano declarado; tipos ajustados al dominio.
Diseño físico	20	Importes en DECIMAL; índices sobre claves naturales y foráneas; particionamiento coherente con el patrón de consulta; justificación escrita de cada decisión.
Carga y SCD Tipo 2	20	Búsqueda de claves sensible a la fecha con BETWEEN; LEFT JOIN con COALESCE hacia las filas especiales; ninguna clave natural con dos versiones vigentes; conteos de dimensiones correctos.

Cuadros de control	15	Los siete controles ejecutados, con captura y con una línea de interpretación por cada uno. El control 4 distingue el problema de calidad de la situación legítima del negocio.
Modelo semántico	10	Esquema estrella verificado; relaciones uno a varios con filtro simple; tabla de fechas marcada; columnas técnicas ocultas; las ocho medidas creadas y el total cuadrando con el origen.
Operaciones OLAP	5	Las seis operaciones resueltas en SQL, más ROLLUP con GROUPING y la versión comentada con GROUPING SETS.
Actividad propuesta	5	Respuesta a la Gerencia en lenguaje de negocio, demostración con datos e indicador de efectividad promocional correctamente construido.

Tabla 12. Rúbrica del laboratorio (100 puntos).

14.1. Penalizaciones

- Scripts que no se ejecutan de principio a fin sobre una base de datos vacía: se descuenta el 50 % del criterio afectado.
- Cuadros de control presentados sin interpretación, solo con la captura: no se otorga puntaje en ese criterio.
- Uso de FLOAT o DOUBLE para importes monetarios: se descuentan 5 puntos del criterio de diseño físico.
- Ratios almacenados como columna en la tabla de hechos o como columna calculada en Power BI: se descuentan 5 puntos.
- Entrega con archivos faltantes o con nombres distintos de los indicados en la tabla 1: se descuentan 3 puntos por archivo.

15. Anexo: solución de problemas frecuentes

Síntoma	Causa probable	Solución
Error 1290: The MySQL server is running with the --secure-file-priv option	La ruta del archivo no está dentro de la carpeta autorizada.	Copie los CSV a la carpeta que devuelve SHOW VARIABLES LIKE 'secure_file_priv', o use la variante LOCAL INFILE.
Error 3948: Loading local data is disabled	local_infile está desactivado en el servidor o en el cliente.	SET GLOBAL local_infile = 1 y active la opción correspondiente en el cliente gráfico.
Los conteos de staging no coinciden con la tabla 4	Terminadores de línea de Windows, o el archivo se truncó al descargar.	Pruebe LINES TERMINATED BY '\r\n' y vuelva a verificar el número de líneas del CSV.
Error 1503 al crear fact_venta	La columna de partición no forma parte de la clave primaria.	tiempo_key debe encabezar la PRIMARY KEY, tal como se indica en la figura 7.
El control 1 muestra menos filas en el hecho que en el origen	Se usó INNER JOIN hacia alguna dimensión, o falta alguna fecha en dim_tiempo.	Revise que los cuatro joins de dimensión sean LEFT JOIN y ejecute el control 7.
El control 5 devuelve filas duplicadas	La carga de hechos se ejecutó dos veces.	TRUNCATE TABLE fact_venta y vuelva a ejecutar dm_mercandina_carga.sql una sola vez.
El control 6 devuelve filas	Se ejecutó dos veces el script de SCD Tipo 2, o falta el WHERE es_vigente = 1.	Recree la base de datos y ejecute los tres primeros scripts una sola vez, en orden.
El importe del hecho no cuadra con el origen	Se usó FLOAT, o se calculó el importe en lugar de tomar importe_netto.	Use DECIMAL(18,2) y cargue directamente la columna importe_netto del origen.

Power BI no ofrece el conector MySQL	Falta el proveedor de datos.	Instale MySQL Connector/NET desde el sitio oficial de MySQL y reinicie Power BI Desktop.
Las funciones de inteligencia de tiempo dan resultados extraños	dim_tiempo no está marcada como tabla de fechas.	Seleccione la tabla, use «Marcar como tabla de fechas» e indique la columna fecha.

Tabla 13. Problemas frecuentes y su solución.

16. Referencias

- Kimball, R. y Ross, M. (2013). The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling (3.ª ed.). Wiley. Capítulos 3 y 5.
- Kimball, R. y Caserta, J. (2004). The Data Warehouse ETL Toolkit. Wiley. Capítulos 5 y 6.
- Pérez, M. (2015). Business Intelligence: Técnicas, herramientas y aplicaciones. Alfaomega.
- Atehortúa, D. et al. (2018). Inteligencia de Negocios con Excel y Power BI. Bii Soluciones & Excel Free Blog.
- Russo, M. y Ferrari, A. (2019). The Definitive Guide to DAX (2.ª ed.). Microsoft Press.
- Documentación de MySQL 8.0: LOAD DATA Statement, Partitioning y WITH ROLLUP.
- Material teórico de la Semana 3 de la asignatura, apartados 3 a 6.